

container-images

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

이 문서는 container-images 문서의 시작점이다. **설계**는 이미지 variant manifest, Dockerfile, entrypoint 구성과 **user-lifecycle** 과의 계약을 설명하고, **운영**은 빌드·배포에 사용하는 명령, 테스트 전략과 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, manifest 기반 variant, Dockerfile/entrypoint 구성, user-lifecycle 계약
운영	빌드/배포 흐름, 테스트 전략, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이미지가 무엇을 책임지고 무엇을 책임지지 않는지 먼저 확인하고, 실제로 빌드하거나 새 variant를 추가해야 할 때 **운영 문서**로 이동한다.

container-images 설계

개요 · 운영

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

1. 책임 범위

container-images 는 여러 CUDA/TensorFlow 조합을 동일한 사용자 경험으로 제공한다. 이미지가 책임지는 것은 OS·CUDA·Python/Jupyter 패키지와 컨테이너 안의 계정, SSH, 선택적 VNC, Kerberos ccache 사용 환경이다.

이미지는 UID/GID를 결정하지 않는다. 사용자 identity, 포트, 홈 디렉터리와 Kerberos keytab은 **user-lifecycle** 와 호스트가 관리한다. 이 경계를 지켜야 이미지가 사용자 DB나 특정 서버에 종속되지 않는다.

2. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 variant가 공유하는 단일 이미지 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver, alias의 권위 있는 목록
<code>entrypoint.sh</code>	실행 시 사용자·그룹·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	이미지 메타데이터, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	<code>user-lifecycle</code> CLI와의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 테스트
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

3. 핵심 기능

3.1 manifest 기반 이미지 variant

`image-variants.json` 한 곳에 다음 항목을 둔다.

- variant ID와 base NVIDIA CUDA image
- CUDA, TensorFlow, Python, Ubuntu 버전
- TensorFlow 설치 패키지와 conda 패키지 제약
- 필요한 최소 NVIDIA driver
- `stable` 또는 `experimental` 지원 상태
- 날짜가 없는 사용자용 alias

현재 안정 계열은 CUDA 11.8, 12.2, 12.3, 12.5이며 CUDA 12.8/H200 계열은 실장비 검증 전까지 experimental이다. 빌드 도구와 GitHub Actions가 같은 manifest를 읽으므로 로컬과 CI의 variant가 달라지지 않는다.

3.2 단일 Dockerfile

`Dockerfile`은 `BASE_IMAGE`, `CUDA_VERSION`, `TENSORFLOW_VERSION`, `MIN_NVIDIA_DRIVER`, `CONDA_PACKAGES` 등을 build argument로 받는다. 모든 variant는 공통으로 SSH, Kerberos client, 한국어 글꼴/입력기, Chrome, Xfce, TigerVNC/noVNC, Miniforge, Jupyter를 포함한다.

variant별 Dockerfile을 복제하지 않은 이유는 보안 패치나 공통 패키지 변경을 한 번만 적용하고, 차이는 manifest의 데이터로 검토하기 위해서다. 버전별 예외가 필요하면 먼저 manifest 값으로 표현하고, 정말 다른 설치 흐름일 때만 Dockerfile에 조건을 추가한다.

3.3 실행 시 identity 구성

`entrypoint.sh`의 주요 흐름은 다음과 같다.

1. 이미지 variant와 실제 host driver 정보를 출력한다.
2. `USER_ID`, `UID`, `GID`, `USER_GROUP`을 검증하고 기존 홈의 owner와 맞춘다.
3. supplemental group과 sudo 정책을 구성한다.
4. SSH 로그인과 Jupyter 설정을 만든다.
5. Kerberos 모드이면 전달받은 ccache를 사용자 환경에 연결한다.
6. Jupyter를 시작하고, 요청된 경우에만 VNC/noVNC를 시작한다.

호스트 mount가 `root_squash`인 환경에서는 root가 사용자 홈을 임의로 `chown`할 수 없다. 그래서 이미 결정된 UID/GID로 사용자를 실행하고, 홈에 쓸 수 없는 root 작업을 최소화한다. 사용자 홈의 기존 비밀번호와 VNC password 파일도 재시작 때 보존한다.

3.4 제한적 sudo와 공유 helper

기본 `DECS_USER_SUDO_MODE=restricted`는 패키지 설치에 필요한 제한된 sudo는 허용하지만 UID 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행을 막는다.

사용자 그룹 공유는 관리자 sudo 대신 `group-dir-share` helper로 제공한다. 사용자 자신의 권한으로 디렉토리를 만들고 `2770`과 default ACL을 설정하므로 공유 기능과 identity 격리를 함께 유지한다.

Kerberos keytab과 ccache를 분리한 이유와 container credential 경계는 [Kerberos/NFS의 keytab과 ccache 모델](#)을 따른다.

3.5 GPU 호환성 표시와 강제 모드

이미지는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true`일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

4. 사용자 생명주기와의 계약

`uidctl create-container`는 image 이름과 version tag, 최종 UID/GID, 포트와 home mount를 Docker 환경변수/옵션으로 전달한다. 이미지가 기대하는 주요 값은 다음과 같다.

값	의미
<code>USER_ID</code>	컨테이너 사용자 이름
<code>UID</code> , <code>GID</code>	DB·AD·NFS와 이미 일치해 검증된 숫자 identity
<code>USER_GROUP</code>	primary group 이름
<code>ENABLE_VNC</code>	VNC/noVNC opt-in

값	의미
KRB5CCNAME	컨테이너에 보이는 ccache 경로
DECS_KERBEROS_HOST_KEYTAB	호스트 관리 ticket을 기다리는 모드
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, NFS와 DB가 어긋나는 오류를 조기에 발견할 수 있다.

container-images 운영

개요 · 설계

1. 빌드와 배포 흐름

```
image-variants.json
|
+--> variant_matrix.py --> GitHub Actions matrix --> build/push
|
+--> build_variants.py -----> local build/push
|
+--> test_image_variants.py -----> smoke test
```

전체 명령 확인:

```
cd /home/jy/server_manage/container-images
python3 scripts/build_variants.py --dry-run
```

특정 variant build:

```
python3 scripts/build_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

registry push는 build 결과와 tag 목록을 확인한 뒤 --push 를 추가한다. 날짜 tag는 재현 가능한 배포 지점을 만들고, stable / latest alias는 현재 권장 variant를 가리킨다. 운영 기록에는 alias보다 날짜가 포함된 tag를 남기는 것이 좋다.

2. 테스트 전략

빠른 정적/회귀 테스트:

```
cd /home/jy/server_manage/container-images
bash tests/test_image_build_config.sh
bash tests/test_entrpoint_root_squash.sh
```

이미지 smoke test:

```
python3 scripts/test_image_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

실제 GPU까지 확인할 때만 `--gpu` 를 사용한다. 사용자 생성 계약은 실제 DB를 바꾸지 않는 `dry-run/print` 경로로 검증한다.

```
python3 scripts/test_uid_create_container.py \
  --variant cuda12.5-tf2.20-ubuntu22.04 \
  --print-only
```

3. 변경 가이드

새 variant 추가

1. `image-variants.json` 에 버전, base image와 최소 driver를 추가한다.
2. `python3 scripts/variant_matrix.py` 결과의 tag를 확인한다.
3. build `dry-run`과 shell 회귀 테스트를 실행한다.
4. 실제 image smoke test와 가능하면 대상 GPU host test를 수행한다.
5. 검증 전에는 `support: experimental` 과 명시적인 alias를 사용한다.

공통 런타임 변경

`Dockerfile` 변경과 `entrpoint.sh` 변경을 구분한다. package/파일은 build 시점에 넣고, 사용자 UID나 mount처럼 컨테이너마다 달라지는 항목만 `entrpoint` 에서 처리한다. `entrpoint` 변경은 재시작 시 idempotent한지, 기존 홈의 파일을 덮어쓰지 않는지, `root_squash` 환경에서 동작하는지를 함께 확인한다.

4. 운영 안전 수칙

- 실제 사용자 비밀번호를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.

- `latest` 만 기록하지 말고 장애 분석이 가능한 날짜 tag를 남긴다.
- `experimental variant`를 `stable alias`로 바꾸기 전에 실장비 GPU test를 한다.
- `sudo` 완화는 `ccache`와 `host bind mount`에 대한 우회 가능성을 먼저 검토한다.
- 로컬 source 변경과 이미 registry에 push된 image를 구분해서 진단한다.